Отчёт по лабораторной работе № 6

Управление процессами

Калашникова Дарья Викторовна

Содержание

1 Цель работы 5

2 Задание 6

3 Выполнение лабораторной работы 7

4 Контрольные вопросы 18

5 Выводы 20

Список литературы 21

Список иллюстраций

3.1 Ввод команд 7
3.2 Ввод 7
3.3 Проверка 7
3.4 Перемещение задания 8
3.5 Проверка 8
3.6 Задания 2 и 3 8
3.7 Ввод 8
3.8 Закрытие 8
3.9 Выход 9
3.10 Убитие заданий 9
3.11 Ввод команд 9
3.12 Запущенные процессы 10
3.13 Использование PID 10
3.14 Иерархия процессов 11
3.15 Закрытие оболочки 11
3.16 Запуск 12
3.17 Изменение приоритета 12
3.18 Изменение приоритета 12
3.19 Завершение процессов 12
3.20 Запуск 13
3.21 Запуск 13
3.22 Запуск 13
3.23 Проверка 13
3.24 Перевод и остановка 14
3.25 Перевод 14
3.26 Проверка 14
3.27 Запуск 14
3.28 Проверка 15
3.29 Получение информации 15
3.30 Запуск 15
3.31 Убитие процессов 15
3.32 Посыл сигнала 16
3.33 Запуск 16
3.34 Завершение работы 16
3.35 Запуск 17
3.36 Изменение 17

Список таблиц

1 Цель работы

Получить навыки управления процессами операционной системы

2 Задание

Продемонстрировать навыки управления заданиями операционной системы, навыки управления процессами операционной системы, а также выполнить задания для самостоятельной работы

3 Выполнение лабораторной работы

Получаем полномочия администратора и вводим следующие команды. Мы запустили последнюю команду без & после неё, у нас есть 2 часа, прежде чем мы снова получим контроль над оболочкой. Введем Ctrl + z , чтобы остановить процесс(рис. 3.1).

```
[dvkalashnikova@dvkalashnikova ~]$ su -
Пароль:
[root@dvkalashnikova ~]# sleep 3600 &
[1] 18081
[root@dvkalashnikova ~]# dd if=/dev/zero of=/dev/null &
[2] 18089
[root@dvkalashnikova ~]# sleep 7200
^Z
[3]+  Остановлен    sleep 7200
```

Рис. 3.1: Ввод команд

Вводим команду jobs и видим три задания, запущенные ранее(рис. 3.2).

```
[root@dvkalashnikova ~]# jobs
[1]   Запущен          sleep 3600 &
[2]-  Запущен          dd if=/dev/zero of=/dev/null &
[3]+  Остановлен    sleep 7200
```

Рис. 3.2: Ввод

Для продолжения выполнения задания 3 в фоновом режиме вводим команду bg 3 и проверяем снова все при помощи команды jobs (рис. 3.3).

```
[root@dvkalashnikova ~]# bg 3
[3]+ sleep 7200 &
[root@dvkalashnikova ~]# jobs
[1]   Запущен          sleep 3600 &
[2]-  Запущен          dd if=/dev/zero of=/dev/null &
[3]+  Запущен          sleep 7200 &
```

Рис. 3.3: Проверка

Для перемещения задания 1 на передний план вводим команду fg 1 (рис. 3.4).

```
[root@dvkalashnikova ~]# fg 1
```

Рис. 3.4: Перемещение задания

Вводим Ctrl + c, чтобы отменить задание 1. С помощью команды jobs посмотрим изменения в статусе заданий (рис. 3.5).

```
sleep 3600
^C
[root@dvkalashnikova ~]# jobs
[2]-  Запущен          dd if=/dev/zero of=/dev/null &
[3]+  Запущен          sleep 7200 &
```

Рис. 3.5: Проверка

Делаем тоже самое для отмены заданий 2 и 3 (рис. 3.6).

```
[root@dvkalashnikova ~]# fg 2
dd if=/dev/zero of=/dev/null
^C447120646+0 записей получено
447120646+0 записей отправлено
228925770752 байт (229 GB, 213 GiB) скопирован, 131,015 s, 1,7 GB/s

[root@dvkalashnikova ~]# jobs
[3]+  Запущен          sleep 7200 &
[root@dvkalashnikova ~]# fg 3
sleep 7200
^C
[root@dvkalashnikova ~]# jobs
```

Рис. 3.6: Задания 2 и 3

Открываем второй терминал и под учётной записью своего пользователя введите в нём команду dd if=/dev/zero of=/dev/null & (рис. 3.7).

```
[dvkalashnikova@dvkalashnikova ~]$ dd if=/dev/zero of=/dev/null &
[1] 18185
```

Рис. 3.7: Ввод

Вводим exit, чтобы закрыть второй терминал (рис. 3.8).

```
[dvkalashnikova@dvkalashnikova ~]$ exit
exit
```

Рис. 3.8: Закрытие

На другом терминале под учётной записью своего пользователя запустим top. Мы видим, что задание dd всё ещё запущено. Для выхода из top используем q (рис. 3.9).

```
op - 01:20:35 up 10:31,  2 users,  load average: 1,96, 1,20, 0,89
asks: 327 total,   2 running, 325 sleeping,   0 stopped,   0 zombie
Cpu(s):  6,0 us, 13,3 sy,  0,0 ni, 77,7 id,  0,4 wa,  0,0 hi,  2,7 si,  0,0 st
iB Mem :   7679,7 total,   3705,2 free,   2749,3 used,   1573,9 buff/cache
iB Swap:   4096,0 total,   4096,0 free,      0,0 used.   4930,4 avail Mem

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  4043 dvkalas+  20   0   19,9g 588796 225116 S 129,1   7,5  41:34.30 firefox
 18185 dvkalas+  20   0  220988   1792   1792 R  92,4   0,0   2:36.64 dd
  3126 dvkalas+  20   0 6889820 430236 136684 S  91,1   5,5  74:11.78 gnome-shell
  4301 dvkalas+  20   0 3011428 254352 154808 S  14,9   3,2   2:50.80 Isolated Web Co
  4181 dvkalas+  20   0 2834812 213132 110916 S   6,6   2,7   0:49.84 Privileged Cont
    75 root      20   0       0      0      0 S   6,3   0,0   1:54.42 ksoftirqd/9
 17790 root      20   0       0      0      0 I   5,3   0,0   0:03.72 kworker/9:0-mm_p+
  4380 dvkalas+  20   0 3208532 449448 133140 S   3,6   5,7   9:03.65 Isolated Web Co
 14807 root      20   0       0      0      0 I   3,0   0,0   0:05.55 kworker/8:0-even+
 16454 dvkalas+  20   0 2847788 164156 118896 S   2,6   2,1   0:21.50 file:// Content
    17 root      20   0       0      0      0 I   1,7   0,0   1:33.79 rcu_preempt
 18106 root      20   0       0      0      0 I   1,3   0,0   0:00.17 kworker/8:2-ata_+
  4240 dvkalas+  20   0 2723784 112424  90428 S   0,7   1,4   0:21.60 WebExtensions
 18237 root      20   0  226032   4480   3584 R   0,7   0,1   0:00.09 top
   913 root      20   0   82624   4224   3840 S   0,3   0,1   0:03.79 irqbalance
  1295 root      20   0  483772  31200  15616 S   0,3   0,4   0:31.94 tuned
  3069 dvkalas+  20   0    6952   4096   2432 S   0,3   0,1   0:21.86 dbus-broker
 10162 dvkalas+  20   0 2843852 232424 116992 S   0,3   3,0   0:58.41 Isolated Web Co
 10244 dvkalas+  20   0 2900664 213784 143496 S   0,3   2,7   1:55.06 Isolated Web Co
```

Рис. 3.9: Выход

Вновь запускаем top и в нём используем k, чтобы убить задание dd. После этого выходим из top (рис. 3.10).

```
top - 01:21:26 up 10:32,  2 users,  load average: 2,02, 1,38, 0,97
Tasks: 327 total,   2 running, 325 sleeping,   0 stopped,   0 zombie
%Cpu(s):  2,8 us,  4,5 sy,  0,0 ni, 92,3 id,  0,0 wa,  0,0 hi,  0,4 si,  0,0 st
MiB Mem :   7679,7 total,   3722,9 free,   2731,5 used,   1557,0 buff/cache
MiB Swap:   4096,0 total,   4096,0 free,      0,0 used.   4948,1 avail Mem
PID to signal/kill [default pid = 18185]
   PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 18185 dvkalas+  20   0  220988   1792   1792 R  92,1   0,0   3:23.15 dd
  4043 dvkalas+  20   0   19,9g 578788 222388 S  18,8   7,4  41:44.27 firefox
  3126 dvkalas+  20   0 6891492 431900 138348 S  13,3   5,5  74:19.54 gnome-shell
  4301 dvkalas+  20   0 2992348 245436 136012 S   4,1   3,1   2:54.10 Isolated Web Co
```

Рис. 3.10: Убитие заданий

Получаем полномочия администратор и вводим следующие команды (рис. 3.11).

```
[root@dvkalashnikova ~]# su -
[root@dvkalashnikova ~]# dd if=/dev/zero of=/dev/null &
[1] 18308
[root@dvkalashnikova ~]# dd if=/dev/zero of=/dev/null &
[2] 18315
[root@dvkalashnikova ~]# dd if=/dev/zero of=/dev/null &
[3] 18324
```

Рис. 3.11: Ввод команд

Вводим ps aux | grep dd. Это команда показывает все строки, в которых есть буквы dd. Запущенные процессы dd идут последними (рис. 3.12).

```
[root@dvkalashnikova ~]# ps aux | grep dd
root           2  0.0  0.0      0     0 ?        S    сен23   0:01 [kthreadd]
dvkalas+    3301  0.0  0.3 955596 30560 ?        Ssl  сен23   0:00 /usr/libexec/evoluti
on-addressbook-factory
dvkalas+    5200  0.0  0.6 287924 47616 ?        Sl   сен23   0:00 /usr/lib64/firefox/f
irefox -contentproc -parentBuildID 20250725060637 -prefsLen 34268 -prefMapSize 249899 -
appDir /usr/lib64/firefox/browser {4ecfa7b4-191b-440c-8726-8ba50f7449e0} 4043 rdd
root       18308 92.2  0.0 220988  1792 pts/0    R    01:22   0:32 dd if=/dev/zero of=/
dev/null
root       18315 90.1  0.0 220988  1792 pts/0    R    01:22   0:23 dd if=/dev/zero of=/
dev/null
root       18324 92.9  0.0 220988  1792 pts/0    R    01:22   0:11 dd if=/dev/zero of=/
dev/null
root       18329  0.0  0.0 221820  2560 pts/0    R+   01:22   0:00 grep --color=auto dd
```

Рис. 3.12: Запущенные процессы

Используем PID одного из процессов dd, чтобы изменить приоритет (рис. 3.13).

```
[root@dvkalashnikova ~]# renice -n 5 18308
18308 (process ID) old priority 0, new priority 5
```

Рис. 3.13: Использование PID

Введем команду ps fax | grep -B5 dd. Параметр -B5 показывает соответствующие запросу строки, включая пять строк до этого. Поскольку ps fax показывает иерархию отношений между процессами, мы также увидим оболочку, из которой были запущены все процессы dd, и её PID (рис. 3.14).

```
sr/lib64/firefox/browser/omni.ja -appDir /usr/lib64/firefox/browser {bb4d43b1-a
973-421d-bc52-cfd1da70bd69} 4043 tab
   4301 ?        Sl     3:03  |   |   \_ /usr/lib64/firefox/firefox -contentpro
c -childID 3 -isForBrowser -prefsLen 31417 -prefMapSize 249899 -jsInitLen 23491
2 -parentBuildID 20250725060637 -greomni /usr/lib64/firefox/omni.ja -appomni /u
sr/lib64/firefox/browser/omni.ja -appDir /usr/lib64/firefox/browser {8e91d562-1
567-47ee-aba1-8204fa011bad} 4043 tab
   4333 ?        Sl     0:02  |   |   \_ /usr/lib64/firefox/firefox -contentpro
c -parentBuildID 20250725060637 -sandboxingKind 0 -prefsLen 34161 -prefMapSize
249899 -appDir /usr/lib64/firefox/browser {8bba1161-59b4-4710-ae55-e83c9aa43215
} 4043 utility
   4380 ?        Sl     9:09  |   |   \_ /usr/lib64/firefox/firefox -contentpro
c -childID 4 -isForBrowser -prefsLen 31524 -prefMapSize 249899 -jsInitLen 23491
2 -parentBuildID 20250725060637 -greomni /usr/lib64/firefox/omni.ja -appomni /u
sr/lib64/firefox/browser/omni.ja -appDir /usr/lib64/firefox/browser {f386e2e1-7
eea-4fea-a7b0-0fc44773ee79} 4043 tab
   5200 ?        Sl     0:00  |   |   \_ /usr/lib64/firefox/firefox -contentpro
c -parentBuildID 20250725060637 -prefsLen 34268 -prefMapSize 249899 -appDir /us
r/lib64/firefox/browser {4ecfa7b4-191b-440c-8726-8ba50f7449e0} 4043 rdd
--
   3273 ?        Ssl    0:14  \_ /usr/libexec/goa-identity-service
   3274 ?        Ssl    0:01  \_ /usr/libexec/evolution-calendar-factory
   3279 ?        Ssl    0:00  \_ /usr/libexec/gvfs-gphoto2-volume-monitor
   3286 ?        Ssl    0:00  \_ /usr/libexec/gvfs-goa-volume-monitor
   3299 ?        Ssl    0:00  \_ /usr/libexec/dconf-service
   3301 ?        Ssl    0:00  \_ /usr/libexec/evolution-addressbook-factory
--
  17909 pts/0    Ss     0:00      \_ bash
  18038 pts/0    S      0:00          \_ su -
  18050 pts/0    S      0:00              \_ -bash
  18272 pts/0    S      0:00                  \_ su -
  18275 pts/0    S      0:00                      \_ -bash
  18308 pts/0    RN     3:54                          \_ dd if=/dev/zero of=/de
v/null
  18315 pts/0    R      3:45                          \_ dd if=/dev/zero of=/de
v/null
  18324 pts/0    R      3:33                          \_ dd if=/dev/zero of=/de
v/null
  18401 pts/0    R+     0:00                          \_ ps fax
  18402 pts/0    S+     0:00                          \_ grep --color=auto -B5
dd
```

Рис. 3.14: Иерархия процессов

Найдем PID корневой оболочки, из которой были запущены процессы dd, и введем kill -9 . Мы увидим, что наша корневая оболочка закрылась, а вместе с ней и все процессы dd (рис. 3.15).

```
[root@dvkalashnikova ~]# kill -9 18275
Убито
```

Рис. 3.15: Закрытие оболочки

Запустим команду dd if=/dev/zero of=/dev/null трижды как фоновое задание (рис. 3.16).

```
[root@dvkalashnikova ~]# dd if=/dev/zero of=/dev/null &
[1] 18592
[root@dvkalashnikova ~]# dd if=/dev/zero of=/dev/null &
[2] 18596
[root@dvkalashnikova ~]# dd if=/dev/zero of=/dev/null &
[3] 18597
```

Рис. 3.16: Запуск

Увеличим приоритет одной из этих команд, используя значение приоритета –5 (рис. 3.17).

```
[root@dvkalashnikova ~]# renice -n -5 18592
18592 (process ID) old priority 0, new priority -5
```

Рис. 3.17: Изменение приоритета

Изменим приоритет того же процесса ещё раз, но используем на этот раз значение –15 (рис. 3.18).

```
[root@dvkalashnikova ~]# renice -n -15 18592
18592 (process ID) old priority -5, new priority -15
```

Рис. 3.18: Изменение приоритета

Завершим все процессы dd, которые мы запустили (рис. 3.19).

```
[root@dvkalashnikova ~]# fg 1
dd if=/dev/zero of=/dev/null
^C312957925+0 записей получено
312957925+0 записей отправлено
160234457600 байт (160 GB, 149 GiB) скопирован, 103,737 s, 1,5 GB/s

[root@dvkalashnikova ~]# fg 2
dd if=/dev/zero of=/dev/null
^C299685188+0 записей получено
299685187+0 записей отправлено
153438815744 байт (153 GB, 143 GiB) скопирован, 100,984 s, 1,5 GB/s

[root@dvkalashnikova ~]# fg 3
dd if=/dev/zero of=/dev/null
^C311365501+0 записей получено
311365500+0 записей отправлено
159419136000 байт (159 GB, 148 GiB) скопирован, 104,804 s, 1,5 GB/s

[root@dvkalashnikova ~]# jobs
[root@dvkalashnikova ~]#
```

Рис. 3.19: Завершение процессов

Запустим программу yes в фоновом режиме с подавлением потока вывода (рис. 3.20).

```
[root@dvkalashnikova ~]# yes > /dev/null &
[1] 18873
```

Рис. 3.20: Запуск

Затем запустим программу yes на переднем плане с подавлением потока вывода. Приостановим выполнение программы. Заново запустим программу yes с теми же параметрами, затем завершим её выполнение (рис. 3.21).

```
[root@dvkalashnikova ~]# yes > /dev/null
^Z
[2]+  Остановлен    yes > /dev/null
[root@dvkalashnikova ~]# yes > /dev/null
^C
```

Рис. 3.21: Запуск

Запустим программу yes на переднем плане без подавления потока вывода. Приостановим выполнение программы. Заново запустим программу yes с теми же параметрами, затем завершим её выполнение (рис. 3.22).

```
y
y
y
y
y
^Z
[3]+  Остановлен    yes
[root@dvkalashnikova ~]# yes
```

Рис. 3.22: Запуск

Проверим состояния заданий, воспользовавшись командой jobs (рис. 3.23).

```
[root@dvkalashnikova ~]# jobs
[1]   Запущен          yes > /dev/null &
[2]-  Остановлен    yes > /dev/null
[3]+  Остановлен    yes
```

Рис. 3.23: Проверка

Далее переведем процесс, который у нас выполняется в фоновом режиме, на передний план, затем остановим его (рис. 3.24).

```
[root@dvkalashnikova ~]# fg 1
yes > /dev/null
^Z
[1]+  Остановлен    yes > /dev/null
```

Рис. 3.24: Перевод и остановка

Переведите любой ваш процесс с подавлением потока вывода в фоновый режим. (рис. 3.25).

```
[root@dvkalashnikova ~]# bg 2
[2] yes > /dev/null &
```

Рис. 3.25: Перевод

Проверим состояния заданий, воспользовавшись командой jobs. Обратим внимание, что процесс стал выполняющимся в фоновом режиме (рис. 3.26).

```
[root@dvkalashnikova ~]# jobs
[1]+  Остановлен    yes > /dev/null
[2]   Запущен          yes > /dev/null &
[3]-  Остановлен    yes
```

Рис. 3.26: Проверка

Запустим процесс в фоновом режиме таким образом, чтобы он продолжил свою работу даже после отключения от терминала (рис. 3.27).

```
[root@dvkalashnikova ~]# nohup yes > /dev/null &
[4] 19310
```

Рис. 3.27: Запуск

Закроем окно и заново запустим консоль. Убедимся, что процесс продолжил свою работу (рис. 3.28).

```
exit
выход
```

Рис. 3.28: Проверка

Далее получим информацию о запущенных в операционной системе процессах с помощью утилиты top (рис. 3.29).

```
Tasks: 335 total,   6 running, 329 sleeping,   0 stopped,   0 zombie
%Cpu(s):  5,7 us, 13,2 sy,  1,3 ni, 79,8 id,  0,0 wa,  0,0 hi,  0,0 si,  0,0 st
MiB Mem :   7679,7 total,   3686,8 free,   2757,7 used,   1572,8 buff/cache
MiB Swap:   4096,0 total,   4096,0 free,      0,0 used.   4922,0 avail Mem

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 18315 root      20   0  220988   1792   1792 R  92,5   0,0  39:57.85 dd
 18324 root      20   0  220988   1792   1792 R  92,5   0,0  39:45.57 dd
 18308 root      25   5  220988   1792   1792 R  92,2   0,0  40:02.77 dd
 18881 root      20   0  220948   1792   1792 R  92,2   0,0   2:16.67 yes
 19310 root      20   0  220948   1792   1792 R  92,2   0,0   0:56.33 yes
  4043 dvkalas+  20   0   19,9g 577512 226860 S   1,2   7,3  47:15.50 firefox
  3126 dvkalas+  20   0 6890976 436004 140792 S   0,6   5,5  77:35.05 gnome-shell
    69 root      20   0       0      0      0 S   0,3   0,0   0:01.07 ksoftirqd/8
 19323 dvkalas+  20   0  226044   4480   3584 R   0,3   0,1   0:00.08 top
     1 root      20   0  175220  17836  10848 S   0,0   0,2   0:08.19 systemd
     2 root      20   0       0      0      0 S   0,0   0,0   0:02.24 kthreadd
```

Рис. 3.29: Получение информации

Запустим ещё три программы yes в фоновом режиме с подавлением потока вывода(рис. 3.30).

```
dvkalashnikova@dvkalashnikova ~]$ yes > /dev/null &
1] 19362
dvkalashnikova@dvkalashnikova ~]$ yes > /dev/null &
2] 19367
dvkalashnikova@dvkalashnikova ~]$ yes > /dev/null &
3] 19372
```

Рис. 3.30: Запуск

Убьем два процесса: для одного используем его PID, а для другого — его идентификатор конкретного задания (рис. 3.31).

```
[dvkalashnikova@dvkalashnikova ~]$ fg 1
yes > /dev/null
^C
[dvkalashnikova@dvkalashnikova ~]$ kill -9 19367
[2]-  Убито               yes > /dev/null
```

Рис. 3.31: Убитие процессов

Далее посылаем сигнал 1 процессу, запущенному с помощью nohup, и обычному процессу (рис. 3.32).

```
[dvkalashnikova@dvkalashnikova ~]$ ps aux | grep yes
root       18881 49.6  0.0 220948  1792 pts/0    R    01:49  15:11 yes
root       19310 91.9  0.0 220948  1792 pts/0    R    02:04  13:51 yes
dvkalas+   19372 91.6  0.0 220948  1792 pts/0    R    02:07  11:41 yes
dvkalas+   19648  0.0  0.0 221664  2048 pts/0    R+   02:19   0:00 grep --color=
auto yes
[dvkalashnikova@dvkalashnikova ~]$ sudo kill -1 19310
[dvkalashnikova@dvkalashnikova ~]$ sudo kill -1 19372
[3]+  Обрыв терминальной линии                    yes > /dev/null
```

Рис. 3.32: Посыл сигнала

Запускаем ещё три программы yes в фоновом режиме с подавлением потока вывода (рис. 3.33).

```
[dvkalashnikova@dvkalashnikova ~]$ yes > /dev/null &
[1] 19698
[dvkalashnikova@dvkalashnikova ~]$ yes > /dev/null &
[2] 19704
[dvkalashnikova@dvkalashnikova ~]$ yes > /dev/null &
[3] 19709
```

Рис. 3.33: Запуск

Завершим их работу одновременно, используя команду killall (рис. 3.34).

```
[dvkalashnikova@dvkalashnikova ~]$ killall yes
yes(18881): Операция не позволена
yes(19310): Операция не позволена
[1]   Завершено      yes > /dev/null
[2]-  Завершено      yes > /dev/null
[3]+  Завершено      yes > /dev/null
```

Рис. 3.34: Завершение работы

Запустим программу yes в фоновом режиме с подавлением потока вывода. Используя утилиту nice, запустим программу yes с теми же параметрами и с приоритетом, большим на 5. Второй процесс имеет nice = 5 (меньший приоритет), поэтому его абсолютный приоритет (PRI) будет на 5 единиц меньше, чем у первого процесса с nice = 0 (рис. 3.35).

```
[dvkalashnikova@dvkalashnikova ~]$ yes > /dev/null &
[1] 19739
[dvkalashnikova@dvkalashnikova ~]$ nice -n 5 > /dev/null &
[2] 19753
nice: если задана поправка, должна быть указана команда
По команде «nice --help» можно получить дополнительную информацию.
[2]+  Выход 125          nice -n 5 > /dev/null
[dvkalashnikova@dvkalashnikova ~]$ fg 2
bash: fg: 2: нет такого задания
[dvkalashnikova@dvkalashnikova ~]$ nice -n 5 yes > /dev/null &
[2] 19847
[dvkalashnikova@dvkalashnikova ~]$ ps -l | grep yes
0 R  1000   19739   17909 91  80   0 - 55237 -      pts/0    00:04:32 yes
0 R  1000   19847   17909 90  85   5 - 55237 -      pts/0    00:00:46 yes
```

Рис. 3.35: Запуск

Используя утилиту renice, изменим приоритет у одного из потоков yes таким образом, чтобы у обоих потоков приоритеты были равны (рис. 3.36).

```
[dvkalashnikova@dvkalashnikova ~]$ renice -n 5 19739
19739 (process ID) old priority 0, new priority 5
[dvkalashnikova@dvkalashnikova ~]$ ps -l | grep yes
0 R  1000   19739   17909 91  85   5 - 55237 -      pts/0    00:05:29 yes
0 R  1000   19847   17909 91  85   5 - 55237 -      pts/0    00:01:44 yes
```

Рис. 3.36: Изменение

4 Контрольные вопросы

1. Какая команда даёт обзор всех текущих заданий оболочки?

Ответ: jobs

2. Как остановить текущее задание оболочки, чтобы продолжить его выполнение в фоновом режиме?

Ответ: Ctrl+Z (остановить), затем bg (продолжить в фоне)

3. Какую комбинацию клавиш можно использовать для отмены текущего задания оболочки?

Ответ: Ctrl+C

4. Необходимо отменить одно из начатых заданий. Доступ к оболочке, в которой в данный момент работает пользователь, невозможен. Что можно сделать, чтобы отменить задание?

Ответ: Использовать команду kill с идентификатором процесса (PID)

5. Какая команда используется для отображения отношений между родительскими и дочерними процессами?

Ответ: pstree

6. Какая команда позволит изменить приоритет процесса с идентификатором 1234 на более высокий?

Ответ: renice -n -10 1234 (уменьшить значение nice для повышения приоритета)

7. В системе в настоящее время запущено 20 процессов dd. Как проще всего остановить их все сразу?

Ответ: killall dd

8. Какая команда позволяет остановить команду с именем mycommand?

Ответ: pkill mycommand

9. Какая команда используется в top, чтобы убить процесс?

Ответ: k (затем ввести PID процесса)

10. Как запустить команду с достаточно высоким приоритетом, не рискуя, что не хватит ресурсов для других процессов?

Ответ: nice -n -10 команда (запуск с высоким приоритетом, но не экстренным)

5 Выводы

В результате выполнения лабораторной работы я получила нывыки работы управления заданиями и процессами операционной системы

Список литературы